Brainy — Étude Métier, Audit Réaliste & Fusion 1.0 / 1.1

Transcription complète du dialogue d'architecture et de stratégie produit · 27 octobre 2026

DEMANDE 1 — UTILISATEUR

Ceci c'est l'application Brainy. J'aimerais que tu l'étudies et que tu me sortes sa philosophie métier, ses caractéristiques, ses applications et ses utilisations dans différents contextes variés.

RÉPONSE 1 — ANALYSE PRODUIT & MÉTIER

1. Philosophie Métier : « Un conteneur unique, une structure choisie, une complexité progressive »

L'objectif de Brainy est d'aider une personne à noter, organiser, relier et retrouver ses informations personnelles sur mobile, ordinateur et web, sans l'enfermer dans un logiciel rigide. Sa philosophie repose sur 5 piliers :

- 1. Un seul modèle de collection (pas de silos figés) : Brainy refuse d'enfermer une collection dès sa création dans un type permanent (« agenda », « budget », « bloc-notes », « inventaire »). Une collection est un conteneur unique qui peut commencer par de simples notes et accueillir plus tard des tâches, des mesures chiffrées, des calculs ou un calendrier selon l'évolution du besoin.
- 2. Séparation stricte entre Structure, Valeur et Exploitation : Le format de données (data format) définit la structure (champs, types, unités) et la couleur pastel. L'entrée enregistre le fait ou la note. La vue ou la règle (liste, tableau, calendrier, formule, analyse, graphique, rappel) exploite et présente les entrées sans dupliquer la donnée source.
- 3. Progressivité (du novice à l'expert) : Un débutant peut créer une collection avec un seul champ texte et écrire immédiatement. Un utilisateur avancé peut définir des unités personnalisées, des objets réutilisables, des références typées, des objets-variables et des formules arithmétiques.
- 4. Intégrité, traçabilité et préservation de l'historique : Un champ contenant des valeurs enregistrées ne peut pas être supprimé ni changer de type ou d'unité. Lorsqu'une entrée référence un objet (ex. un produit avec un prix), Brainy distingue le mode valeur actuelle du mode instantané / valeur figée (qui préserve les montants historiques). Les calculs vérifient la compatibilité des types et des unités.
- 5. Portée locale et partage explicite : Chaque format de données, unité locale ou objet appartient d'abord à sa collection. Sa réutilisation dans une autre collection (copie, partage lié ou déplacement) résulte toujours d'un choix explicite de la cible avec contrôle des dépendances.

2. Caractéristiques Clés de l'Application

A. Modèle de données et structuration

- Collections & Modèles de départ : Création libre ou à partir de modèles de départ modifiables.
- Formats de données (1 à 24 par collection) : Chacun possède son nom, sa version et une couleur pastel unique au sein de la collection.
- Champs riches et typés (1 à 30 par format) : Titre, texte court, texte long, caractère unique, lien web, nombre (sans unité, unité proposée ou unité locale ; support de la virgule française), date, heure, date et heure, durée, oui/non, choix unique/multiple, pièce jointe (2 Mio/fichier, 8 Mio/entrée), référence d'objet et liens entre entrées.
- Modèles d'objets et Objets réutilisables : Un format de données marqué comme modèle d'objets produit des objets nommés avec identité stable, historique de modifications, duplication et utilisation via objets-variables dans les calculs.

B. Exploitation, Vues, Analyses et Architecture

- Vues sauvegardées & Rappels : Liste compacte, tableau, vues Tâches (état, priorité, échéance), Agenda / calendrier et rappels liés aux champs de date/heure.
- Formules, Analyses, Graphiques, Stocks et Prévisions : Calculs arithmétiques par entrée (quantité × produit.prix), analyses multi-entrées (somme, compte, moyenne, min, max), graphiques, registre local de mouvements de stock (T26) et premières prévisions explicables (T27).
- Expérience & Écosystème : Direction visuelle inspirée de Fedora Linux (thèmes clair et sombre charbon, fonds pastel), bilinguisme intégral FR/EN, Web Next.js (brainy_web_app/), Mobile & Desktop Flutter (brainy/), API GraphQL NestJS + MongoDB (api/).

3. Applications et Utilisations dans Différents Contextes

- 1. Organisation personnelle & Productivité (PKM, To-Do, Agenda) : Combiner notes rapides, signets web, documents joints, tâches prioritaires et rendez-vous dans une même collection de projet.
- 2. Finances personnelles et Gestion de budget : Objets Commerçant/Abonnement, entrées Dépense en EUR, analyses mensuelles par catégorie avec prix historiques figés.
- 3. Santé personnelle, Sport et Habitudes : Journal privé de constantes (poids en kg, pouls en bpm, sommeil) et d'entraînements (distance en km) avec graphiques d'évolution.
- 4. Recherche, Sciences et Relevés de terrain : Objets Station ou Échantillon et relevés horodatés (température en °C, pluviométrie en mm) avec contrôle strict des unités.
- 5. Artisanat, Petite activité commerciale et Boutique : Catalogue d'articles, calcul automatique des ventes et poids de colis, suivi des stocks par mouvements validés (T26).
- 6. Éducation, Études et Écriture de livre : Chapitres, scènes, notes et citations avec ordre de manuscrit, liens internes et export texte lisible (T25).

DEMANDE 2 — UTILISATEUR

Maintenant donne-moi ton avis réaliste sur l'application.

RÉPONSE 2 — AUDIT RÉALISTE (PRODUIT & TECHNIQUE)

1. Les vraies forces (Ce qui distingue vraiment Brainy)

- Modélisation métier supérieure à Notion/Airtable sur deux points : (1) La distinction native entre valeur actuelle et instantané figé (T13), qui évite qu'un changement de tarif catalogue ne modifie les ventes passées. (2) Le typage des nombres par unité et l'arithmétique sûre (unit_math.ts, T08, T21), qui empêche d'additionner des EUR et des km.
- Collection à formats multiples et code couleur pastel : Permettre jusqu'à 24 formats de données dans une même collection offre la souplesse d'un carnet papier avec la rigueur d'une base structurée.
- Discipline d'ingénierie : Parité stricte API GraphQL + Next.js + Flutter, bilinguisme FR/EN et tests de fumée automatisés.

2. Les risques Produit, UX et Techniques identifiés

- Risque Produit — Positionnement : Éviter de donner l'impression d'un ERP industriel ou médical certifié ; affirmer Brainy comme un établi de données structurées.
- Risque UX — Jargon et Onboarding : Le questionnaire de profil exigeant un centre d'intérêt ou loisir (pour T33 non livrée) crée de la friction à l'inscription. Les concepts d'objet-variable doivent rester masqués derrière des formulations naturelles.
- Limite Technique — Fichiers monolithiques : Dans Flutter, workspace_screen.dart dépasse 4 470 lignes (et collections.service.ts ~1 118 lignes côté API). Continuer à empiler des fonctionnalités dans ces mêmes fichiers ralentira le développement et augmentera la consommation de crédits IA.

Les 3 recommandations concrètes formulées :

• 1. Prioriser le suivi structuré quotidien, scientifique et artisanal avant T32 (Industriel) et T33 (Social).
• 2. Alléger l'onboarding (profil facultatif avant T33) et masquer le jargon technique dans l'interface.
• 3. Stopper la croissance de workspace_screen.dart en isolant chaque nouvelle vue ou capacité dans des fichiers dédiés.

DEMANDE 3 — UTILISATEUR

Ce n'est pas que je veux couvrir tous les domaines comme des applications séparées, mais créer une philosophie et des principes qui permettent aux gens de représenter, manipuler et toucher tout type de données. Prends en compte tes 3 recommandations, lis ROADMAP_BRAINY_1_1.md et dis-moi honnêtement comment joindre tes recommandations et ma vision 1.1.

RÉPONSE 3 — ANALYSE DE ROADMAP_BRAINY_1_1.MD & STRATÉGIE DE FUSION

1. Mon avis honnête sur ROADMAP_BRAINY_1_1.md

Votre clarification change tout : vous ne cherchez pas à coder 50 logiciels métiers différents, mais les 12 primitives universelles de la donnée qui manquent aujourd'hui pour modéliser le réel sans tricher.

- L'épreuve des 50 domaines (Annexe A) est remarquable : Elle identifie exactement pourquoi la donnée réelle résiste aux outils classiques : besoin de capture en 2–3 gestes (T34), d'opérations sur séquences et distributions (T35), d'un vrai champ mesure avec incertitude, statut et révisions (T36), de relations hiérarchiques et compositions (T37), de géographie sur choix explicite (T38), de classifications (T39), de flux d'états (T40), de collecte tierce en quarantaine (T41), de registre non modifiable (T42), de seuils de confidentialité (T43), de fichiers lourds S3 (T44) et de paquets de domaine (T45).
- Le positionnement Hors Périmètre (Sections 1 & 9) : Pose noir sur blanc que Brainy ne promet jamais de conformité légale, médicale, fiscale ou industrielle.
- Convergence totale avec les Ajustements A1 à A10 : Votre Section 5 anticipait déjà l'ajustement A1 (profil facultatif avant T33), A2 (test terminologique de format de données), A4 (registre d'opérations) et A5 (unification des stocks T26 et du registre T42).

DEMANDE 4 — UTILISATEUR

Sachant que je suis à T27 déjà exécutée, modifie AGENTS.md, ROADMAP_BRAINY.md et COLLECTIONS_DIRECTION.md pour intégrer la fusion de tes recommandations et ma vision 1.1, avec une dépense minimale de crédits IA et une structure permettant à 2 IA ou plus de travailler sans conflit. Dis-moi ensuite à quelle tâche continuer.

RÉPONSE 4 — STRUCTURATION DU DÉPÔT, ÉCONOMIE DE JETONS & ORDRE DE SUITE

1. Modifications appliquées au dépôt

- AGENTS.md (~11,7 Kio, 77 lignes) : Mis à jour avec l'état livré jusqu'à T27 (stocks T26 et prévisions T27), le résumé 1.1 (T34–T45), la règle anti-monolithe obligatoire et le protocole de coordination Multi-IA.
- ROADMAP_BRAINY.md & Fiches T34.md à T45.md : Découpage des 12 tâches 1.1 en fiches individuelles (< 2 Kio chacune) dans docs/tasks/T34.md à T45.md ; archivage des 50 domaines et scénarios R1–R10 dans docs/ANNEXE_50_DOMAINES.md ; ajout des règles 11–18 et ajustements A1–A10 dans docs/tasks/_PRINCIPES.md ; ajout de l'Ordre d'exécution fusionné en 3 vagues.
- COLLECTIONS_DIRECTION.md & docs/ETAT_IMPLEMENTATION.md : Alignés sur T01–T27 livrés, tableau des 12 primitives 1.1 et décisions produit ouvertes (A1, A2, T42).
- docs/REPRISE_CODEX.md (1,1 Kio) & scripts/etat-reprise.mjs : Journal ramené de 13 Kio à 1,1 Kio avec

verrou de tâche/fichiers réservés, et script inspectant les 4 dépôts Git (., api, brainy_web_app, brainy) en un seul appel.

2. À quelle tâche continuer maintenant ?

- Option A — Avec 1 seule IA (ou 2 IA à tour de rôle) : Continuer par T34 — Capture rapide et saisie rapide (dépendances T06, T24, T25 déjà terminées ; gain quotidien immédiat sur les scénarios R1 et R2 en isolant les composants UI).
- Option B — Avec 2 IA en parallèle (Zéro conflit de fichiers) : IA n°1 sur T34 (Capture & saisie rapide côté UI : quick-capture-dialog.tsx, quick_capture_sheet.dart) et IA n°2 sur T35 (Registre d'opérations d'analyse extensibles côté backend/analyse : analysis_registry.ts), qui débloque immédiatement T36 (Champ « mesure »).
- Séquence complète recommandée : Vague 1 : T34 -> T35 -> T36 -> T37 -> T38 -> T39 | Vague 2 : T28 -> T29 -> T40 -> T42 -> T43 -> T30 -> T41 -> T31 -> T44 | Vague 3 : T45 -> T32 -> T33.

DEMANDE 5 — UTILISATEUR

Ça veut dire qu'il n'y avait pas de conflit entre les nouvelles directives et les anciennes de T1 à T27 ?

RÉPONSE 5 — LES 6 ÉCARTS RÉELS ENTRE LE CODE T01–T27 ET LES DIRECTIVES 1.1

Sur la philosophie métier, il n'y a aucun conflit destructeur : rien de T01 à T27 n'est à jeter. En revanche, dans le code réel écrit entre T01 et T27, il existe 6 écarts concrets d'alignement ou de finition prévus par les ajustements A1 à A10 :

- 1. Mesures de T27 vs Champ « mesure » de T36 : Dans T27, les modèles Météo/Science/Santé utilisent plusieurs champs séparés ordinaires (un champ nombre + un champ qualité + un champ source). T36 introduit un seul type de champ composite « mesure » (valeur, unité, statut mesurée/estimée/absente, incertitude, révisions). Lors de T36, on migrera les modèles de T27 vers ce champ et on permettra de convertir un champ nombre existant en mesure sans perte.
- 2. Stocks de T26 vs Registre non modifiable de T42 (Ajustement A5) : Dans T26, les mouvements de stock sont idempotents et traçables, mais codés avant la chaîne de hachage cryptographique de T42. Lors de T42, on branchera la chaîne de hachage et la contre-écriture directement sur le registre de T26.
- 3. Analyses de T22 vs Registre d'opérations de T35 (Ajustement A4) : Dans T22, les 5 opérations (somme, compte, moyenne, min, max) sont codées en dur. En T35, on extraira d'abord cet agrégateur dans un registre d'opérations versionné (analysis_registry.ts) avant d'ajouter médiane, percentiles et différences de compteur.
- 4. Modèles codés en dur (T24/T26/T27) vs Paquets de domaine (T45 / A8) : Les modèles de départ actuels sont définis dans le code ; ils adopteront le format déclaratif et versionné des Paquets de domaine en T45.
- 5. Profil obligatoire (A1) et README en retard (A9) : Le code actuel oblige encore à saisir au moins un centre d'intérêt ou loisir à l'inscription (pour T33 non livrée), et README.md omet encore la référence d'objet, les stocks et les prévisions. Peut être aligné immédiatement.
- 6. Validation UI de T26/T27 et taille de workspace_screen.dart : T26 et T27 ont été clôturées par l'unique test API smoke:stock-forecast sans nouveaux tests de widgets Flutter, et workspace_screen.dart atteint 4 471 lignes. D'où la règle d'isoler tout nouveau composant dans des fichiers dédiés dès T34.